Extending CT-Prover: Detecting Time-Variant Instruction Side-Channel Sources

Zachary Riback
Rochester Institute of Technology
zrr4832@rit.edu

Sam Millman
Rochester Institute of Technology
sam5346@rit.edu

1 Introduction

Cryptographic operations are supposed to run in constant time — meaning their execution doesn't vary depending on the input. Therefore, if execution time changes, an adversary can exploit those differences to leak sensitive information, such as cryptographic keys or secret messages. Our solution is to extend CT-Prover, a taint analysis tool, to detect these timing leaks more effectively. While CT-Prover already checks for some constant-time violations, it misses certain sources, like division instructions. Our project focuses on improving its functionality to catch these additional sources of non-constant time behavior. This extension makes CT-Prover more practical for analyzing real-world cryptographic libraries.

2 Background

To fully understand the contribution to extend CT-Prover, it is necessary to know about the moving pieces behind code analysis and how they are utilized. We discuss the importance of constant time as well as the importance of taint analysis and tainted data control flow analysis. We also discuss side channels and their impact, an area where there is a multitude of literature to study from and learn how to better protect cryptography functions.

Constant time, often written as O(1), means a function's runtime doesn't depend on its inputs. In cryptography, this is critical because inputs often contain secret information. If runtime varies, an adversary can feed different inputs and measure execution time to infer secrets. This explains why constant-time execution is a cornerstone of secure cryptographic design. Even small timing differences can accumulate into devastating leaks when exploited repeatedly.

A side channel is an unintended pathway where information leaks from a system. It's not part of the system's intended design, but it can reveal secrets. For example, CPUs draw more power when performing addition compared to subtraction. An attacker could observe power usage and deduce which operation is happening. Side channels come in many forms — timing, cache, power, and more — usually stemming from shared resources. In our work, we focus specifically on timing differences caused by varying inputs. This focus allows us to target one of the most practical and widely exploited forms of leakage.

A DIV instruction is a low level instruction carried out by a CPU, typical in machine code instructions. In x86 assembly, the DIV instruction is particularly problematic. Unlike some other mathematical functions, division is not constant time. It takes a variable number of clock cycles depending on the operands. This variability introduces timing differences that attackers can exploit. Division instructions are a prime target for detection in our extended CT-Prover. By flagging these cases, we can prevent vulnerabilities that slip past traditional constant-time checks.

Code analysis techniques have developed over the years, and are the basis of how CT-Prover works. Safety Verification is the process of proving that a program or system never reaches what we call a bad state. That could mean preventing things like memory errors, type violations, or unsafe concurrency conditions. In practice, this often translates into safe coding practices and automated checks that help ensure the software behaves reliably and securely. LLVM is a full compiler framework, and one of its most important features is its Intermediate Representation, or IR. IR is like a low-level assembly language, but it's typed and platform-independent. Because of this, LLVM can take code from different languages, optimize it, and then translate it into machine code for many different platforms. It's also extremely useful for analysis, because researchers and tools can inspect programs at the IR level to catch unsafe patterns or optimize performance before code is even compiled down. Finally, Clang is the compiler front end for the C family of languages, and it's built on top of LLVM. Clang is responsible for parsing the source code — handling syntax and semantic analysis — and then translating it into LLVM IR. One of the reasons Clang is so widely adopted is because it not only compiles code efficiently, but it also provides excellent diagnostic messages and libraries that support static analyzers and other developer

tools. It plays an important role in both code optimization and improving the overall software development process.

Taint analysis is a way of studying how untrusted or sensitive data flows through a program. It starts with what's called tainted data — this can include user input, network traffic, or even secret values like cryptographic keys. Taint analysis marks this data and then traces how it moves through variables, functions, and memory. Static taint analysis works by examining the code without actually running it. The advantage here is that it can cover all possible execution paths, making it very thorough. The drawback is that it can sometimes be unreliable, producing false positives, because it has to make conservative assumptions about how the program behaves. Static taint analysis is widely used in compilers, verification tools, and security scanners. Dynamic taint analysis, in contrast, instead looks at the code abstractly, following data during actual program execution. This makes it very precise for the particular run it analyzes, but it's limited to the paths that were actually executed. If a branch isn't triggered in that run, it will not be observed how taint flows through it. For example, taint analysis can be used in the following ways. It can be used in detecting injection attacks (e.g., SQL injection, command injection) by seeing if unsanitized user input reaches a database or system call. It can find side-channel leaks by checking if secret values influence control flow or memory access. Lastly, it can be used in malware analysis, to see how untrusted data from a file or network propagates inside malicious code. In practice, both static and dynamic taint analysis complement each other: static gives broad coverage, while dynamic provides precision during real execution

3 Related Work

Brumley et. al discuss the importance of constant time implementations of cryptographic functions in their study "Remote timing attacks are still practical" [3]. The authors find that timing attacks exist in the real world, specifically identifying a vulnerability resulting in a timing attack in OpenSSL. This vulnerability resulted in a lattice attack with recovered bits leading to a full cryptography key recovery.

CT-Verif is a tool designed to automatically verify constant-time cryptographic implementations [1]. It translates LLVM IR into Boogie, an intermediate verification language. Then solvers like Corral and Z3 check whether code branches depend on secret data. CT-Verif laid the groundwork for CT-Prover, which builds on these ideas with taint-directed self-composition. However, CT-Verif doesn't explicitly handle division instructions as timing-sensitive operations. That gap is what extending CT-Prover benefits from; to detect tainted divisions alongside branches, memory accesses, and loops. In other words, we're broadening the scope of verification to catch vulnerabilities that CT-Verif and CT-Prover would miss.

Bernstein et. al write about their exploitation of secret-dependent divisions in Kyberslash [2]. They observe Kyber is

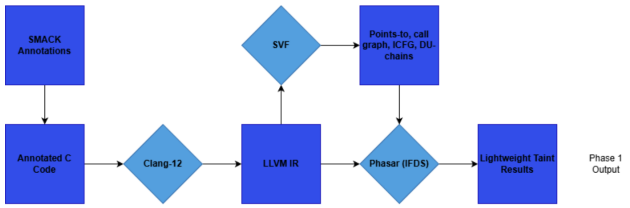

Figure 1: CT-Prover Phase 1 Diagram

a NIST-standardized post-quantum cryptographic mechanism. In one implementation, division instructions were used with secret polynomial coefficients as divisors. Because division time depends on the size of the divisor, larger coefficients cause longer execution times. This created a timing side channel that leaked secret values. By carefully crafting specific strings of ciphertext, a victim could experience a leak of their full cryptography key. KyberSlash is a practical, real-world example of how division instructions can compromise cryptographic security — reinforcing the need for our improvements to CT-Prover. It demonstrates that these vulnerabilities aren't just theoretical, but exploitable in widely deployed cryptographic standards.

From Schröder et al., Divide and Surrender discussed division instructions and their real world impact in cryptographic schemes [5]. This study found that compilers sometimes replace division with Barrett reductions, which are constant time.In certain contexts however, like inside loops, they instead emit DIV instructions. This substitution creates a timing side channel. In fact, it was exploited in HQC, a post-quantum cryptographic scheme, where division instructions leaked secret values. In this paper, the authors noted that they responsibly disclosed the vulnerability to the proper sources, patching it before it was publicized. This shows how compiler behavior can unintentionally introduce vulnerabilities — and why tools like CT-Prover need to detect these cases. It highlights that even trusted compiler optimizations can become dangerous in cryptographic contexts.

4 Baseline

CT-Prover [4], is an improvement to the popular ct-verif [1] tool. CT-Prover seeks improve side-channel source detection methods, and it manages to find more potential side-channel sources in less time when compared to ct-verif over the same benchmarks. While the most in-depth information on CT-Prover can certainly be found in its original paper, phase 1 (out of 3) is worth talking about here because it is the part of CT-Prover that our main contribution relies upon.

The workflow of phase 1 of CT-Prover can be seen in Figure 1. First, SMACK annotations are used to annotate C code in order to tell the taint analysis engine what data is secret. Clang-12 is then used to compile the C code into

LLVM IR. SVF [7] is then used to perform preliminary taint analysis methods, such as points-to analysis and creation of call graphs, ICFGs, and def-use chains. These are fed into the final step utilizing PhASAR [6], which is a LLVM-based static taint analysis engine that produces the phase 1 output, which comes in the form of specific LLVM IR lines that are found to house instructions utilizing tainted data.

5 Contributions

Our main contribution, which was our main idea from the beginning, was to add the capability to CT-Prover to detect tainted divisions. The authors of CT-Prover in the original paper point this out as a potential side channel source that they ignore, but it could be added to CT-Prover's capabilities [4]. Additional motivation for adding tainted division detection is found in KyberSlash [2] and Divide and Surrender [5] papers, which were covered in the related work section. These papers prove that time-variant division operations that use tainted data are an actual exploitable vulnerability that exists in the real world, and so it is worth detecting them.

In addition to this contribution, many other minor improvements were made to CT-Prover's code, which were necessary to even work with the tool in the first place.

5.1 Detecting Tainted Divisions

In order to detect tainted operations, CT-Prover uses the PhASAR [6], an LLVM-based static analysis framework, to perform the static taint analysis. PhASAR locates certain operations, such as branches and data accesses, and then pushes their operands onto a `TaintSeeds` stack in order to track if those operands depend on secret information. If they do, PhASAR is able to record that LLVM IR line as having a violation that could result in a timing-based side channel vulnerability.

In order to add support to PhASAR to detect tainted divisions, all the possible LLVM instructions that invoke integer division must be captured and have their operands pushed onto the stack. The necessary operations to look for are `sdiv`, `udiv`, `fdiv`, `srem`, `urem`, and `frem`.

Figure 2 shows a snippet of the modification done to PhASAR's backend. With this change, PhASAR will now consider the operands of all the aforementioned division/remainder operations as possible taint sources. Both operands of the division (or remainder) operation are pushed onto the `TaintSeeds` structure, meaning PhASAR will be able to raise warnings for those lines if any of those operands are found to depend on secret data at any point during program execution.

5.2 Showing Tainted Source Code Lines

Someone utilizing CT-Prover to check their code would benefit greatly from output that shows *where* in the source code the tainted operations are happening, instead of simply the LLVM IR line. We instrumented CT-Prover to be able to generate the original source code, but with markings showing what source lines, if any, were found to be tainted after the phase 1 analysis. The original C source files, which are present in the benchmark, were regenerated, but with comments showing what lines are tainted branches, data access, and divisions. A sample of this output is shown in Figure 5.

5.3 Other Improvements

While the main goal was to add support for detecting tainted division operations, other improvements had to be naturally made to CT-Prover as a necessity in order to work with the tool.

Fixing Absolute Paths Unfortunately, CT-Prover's code contains hundreds of instances where absolute paths are used. This breaks the code in hundreds of places when run on a new machine. Figure 3 shows one of many examples of this from the script `vfctnew6.py`, which is a critical script in running the analysis pipelines. As can be seen, absolute paths to important PhASAR and Bam binaries/directories are present. All of these will cause errors when running CT-Prover on a machine that is different from the one on which it was originally developed. We fixed this shortcoming by using a `CTPROVER_ROOT` environment variable to allow all these scripts to find the root dynamically. This makes the tool actually possible to use without needing manual modifications. A sample of code showing this change can be seen in Figure 4.

Improved Scripts CT-Prover's scripts leave a lot to be desired. There does not seem to be an easy way to run only the first phase or only all three phases at a time. Instead, the included scripts run all of the original author's tests every time, including only the first phase, only the second phase, only the third phase, just the first two phases, just the second and third phases, and so on. Additionally, there seems to be no way of automatically running the analysis on all the included benchmarks at once. There is also no way to easily analyze the results, including execution time information. This seems to suggest that much of the analysis discussed in the original paper was done manually.

We wrote several scripts to attempt to solve these issues. A summary of these scripts can be found in Table 1. Anyone who is more interested in these scripts should read the associated documentation in the GitHub repository for this project.

```
auto getdumpresult(std::set<llvm::Instruction*>& res, std::set<llvm::Instruction*>& res2, std::set

    std::set<llvm::Module *> mds = IRDB.getAllModules();
    for(auto M : mds){
        for(auto &F : *M)
            for(auto &BB : F)
                for(auto &I : BB){
                    if(llvm::isa<llvm::DbgInfoIntrinsic>(&I))
                        continue;

                    std::vector<const llvm::Value *> TaintSeeds;
                    if(const auto *Load = llvm::dyn_cast<llvm::LoadInst>(&I)){
                        TaintSeeds.push_back(Load->getPointerOperand());
                    } else if(const auto *Store = llvm::dyn_cast<llvm::StoreInst>(&I)){
                        TaintSeeds.push_back(Store->getPointerOperand());
                    }else if(const llvm::BranchInst *Br = llvm::dyn_cast<llvm::BranchInst>(&I)){
                        if (Br->isUnconditional())
                            continue;
                        TaintSeeds.push_back(Br->getCondition());
                    }else if(const auto *BinOp = llvm::dyn_cast<llvm::BinaryOperator>(&I)){
                        switch (BinOp->getOpcode()) {
                        case llvm::Instruction::SDiv:
                        case llvm::Instruction::UDiv:
                        case llvm::Instruction::FDiv:
                        case llvm::Instruction::SRem:
                        case llvm::Instruction::URem:
                        case llvm::Instruction::FRem:
                            TaintSeeds.push_back(BinOp->getOperand(0));
                            TaintSeeds.push_back(BinOp->getOperand(1));
                            break;
                        default:
                            break;
                        }
```

Figure 2: PhASAR modification to detect tainted divisions

Script	Purpose
vfct_all.py	Working runner for executing all analysis pipelines on one benchmark program
vfct_only1.py	Run only the phase 1 analysis on one benchmark program
vfct_123.py	Run all phases of analysis on one benchmark program
vfct_1_all.py	Run only the phase 1 analysis on ALL benchmark programs
vfct_123_all.py	Run all three phases of analysis on ALL benchmark programs
summarize_results.py	Summarize all analysis output and timing information in one easy-read-text file
annotate.py	Helper script for creating annotated C code showing the location of tainted operations

Table 1: Summary of added scripts

```
PHASAR  ="/home/luwei/phasar/build/tools/phasar-llvm/phasar-llvm"
BAMPATH = "/home/luwei/bam-"

...

#record 是bool版验证的结果，  file是shadow product的结果。 file2是最后生成的文件
def transfer(record,file,file2, recordfile = subprocess.PIPE, workdir = os.getcwd()):
    # 需要配合bam991 一起使用
    args = "transBoolToShadow.py "+record+" "+file
    restime = runcommand(args, recordfile, workdir = workdir)
    args = "ruby -I /home/luwei/bam-991/lib /home/luwei/bam-991/bin/bam --process_mark
    restime2 = runcommand(args, workdir=workdir)
    return restime, restime2
```

Figure 3: Absolute paths in vfctnew6.py

```
CTPROVER_ROOT = os.environ["CTPROVER_ROOT"]
PHASAR = os.path.join(CTPROVER_ROOT, "phasar/build/tools/phasar-llvm/phasar-llvm")
BAMPATH = os.path.join(CTPROVER_ROOT, "bam/bam-")

...

#record 是bool版验证的结果，  file是shadow product的结果。 file2是最后生成的文件
def transfer(record,file,file2, recordfile = subprocess.PIPE, workdir = os.getcwd()):
    # 需要配合bam991 一起使用
    args = "transBoolToShadow.py "+record+" "+file
    restime = runcommand(args, recordfile, workdir = workdir)
    args = "ruby -I /home/luwei/bam-991/lib /home/luwei/bam-991/bin/bam --process_mark
    restime2 = runcommand(args, workdir=workdir)
    return restime, restime2
```

Figure 4: Dynamic path finding code

```
static int bad_function(int secret, int public) {
    int sum = secret + public;

    if (sum > 100) {    // TAINTED BRANCH
        printf("sum is greater than 100\n");
    }

    for (int i = 0; i < secret; i++){    // TAINTED BRANCH
        printf("looping!\n");
    }

    char string[] = "Hello there.";
    printf("Char at %d: %c", secret, string[secret]);    // TAINTED DATA ACCESS
    printf("Char at %d: %c", public, string[public]);

    int quotient1 = secret / 10;    // TAINTED DIVISION
    int quotient2 = public / 10;
    int remainder = secret % 10;    // TAINTED DIVISION

    return 0;
}
```

Figure 5: Marked sample output

```
/*************************************************
* Name:        poly_tomsg
*
* Description: Convert polynomial to 32-byte message
*
* Arguments:   - uint8_t *msg: pointer to output message
*              - const poly *a: pointer to input polynomial
**************************************************/
void poly_tomsg(uint8_t msg[KYBER_INDCPA_MSG_BYTES], const poly *a)
{
    unsigned int i,j;
    uint16_t t;

    for(i=0;i<KYBER_N/8;i++) {
        msg[i] = 0;
        for(j=0;j<8;j++) {
            t  = a->coeffs[8*i+j];
            t += ((int16_t)t >> 15) & KYBER_Q;
            t  = (((t << 1) + KYBER_Q/2)/KYBER_Q) & 1;    // TAINTED DIVISION
            msg[i] |= t << j;
        }
    }
}
```

Figure 6: Result of running CT-Prover on vulnerable Kyber code

Improved Documentation Much of the difficulty in working with CT-Prover came from its lack of documentation. Set-up instructions, build instructions, and dependencies were not adequately explained. We updated the main README.md file in the repository to give full environment setup, build, and execution instructions.

6 Results

After adding the capability to detect tainted divisions to CT-Prover, we re-ran the same benchmarks from the original paper in order to discover if adding divisions as a potential side-channel source has a significant impact on the results. A summary of this test is shown in Table 2, with references to the original name for the benchmark from CT-Prover's original paper [4].

In total, eight instances of tainted divisions across three separate programs were found. Given that 70/87 of CT-Prover's original benchmarks are verified constant-time by only the phase 1 analysis (phase 1 analysis finds NO tainted locations), this seems to be an appropriate number of detections that one would expect. Interestingly, one of these results is located in a program that CT-Prover previously deemed constant time after phase 1 analysis.

Unfortunately, due to inexperience with complex program analysis and difficulties working with CT-Prover, we were not able to run the second and third phases of analysis on these taint locations. Even so, these results illustrate how tainted divisions do in fact occur in real cryptographic libraries. We leave the further task of investigating these locations, be it with CT-Prover or manual analysis, to future work.

KyberSlash Additionally, in order to prove that CT-Prover with tainted division detection capabilities is able to highlight division operations that can actually lead to exploitable side-channel attacks, we look to KyberSlash [2], which was

Name	Lib/Algorithm/Function	CT/non-CT	New Divisions Detected
P1_2	OpenSSL_ssl3_cbc_digest_record	CT	3
P2_1	MAC-then-Encode-then-CBC-Encrypt	CT	4
P6_27	BearSSL_RSA_i32_decrypt	non-CT	1

Table 2: Results of detecting tainted divisions in CT-Prover's benchmarks

```
void poly_tomsg(uint8_t msg[KYBER_INDCPA_MSGBYTES], const poly *a)
{
  unsigned int i,j;
  uint16_t t;
  uint32_t t;

  for(i=0;i<KYBER_N/8;i++) {
    msg[i] = 0;
    for(j=0;j<8;j++) {
      t  = a->coeffs[8*i+j];
      t += ((int16_t)t >> 15) & KYBER_Q;
      t  = (((t << 1) + KYBER_Q/2)/KYBER_Q) & 1;
      // t += ((int16_t)t >> 15) & KYBER_Q;
      // t  = (((t << 1) + KYBER_Q/2)/KYBER_Q) & 1;
      t <<= 1;
      t += 1665;
      t *= 80635;
      t >>= 28;
      t &= 1;
      msg[i] |= t << j;
    }
  }
```

Figure 7: Commit that removes tainted division for Kyber

covered previously in the related work section. By pulling out the related code from Kyber that is referenced in the KyberSlash paper, we can annotate the secret data and create a new benchmark in CT-Prover to test this program. The resulting marked file from this analysis is shown in Figure 6. As can be seen, a variable that is dependent on secret polynomial coefficients is used as a division operand, and CT-Prover generates this warning for us and shows exactly on which source line the problem exists. This line ultimately allowed the KyberSlash exploit to exist.

For the sake of completeness, Figure 7 shows the fix put in place by the authors of Kyber to patch KyberSlash (commit hash dda29c). The line that CT-Prover marked has been removed, and the necessary algorithm that Kyber performs is now completed with bit shifting, addition, and multiplication. This illustrates how many tainted divisions can be mitigated: by writing the source code in a way that prevents the compiler from generating integer division instructions.

7 Future Work

There are many more improvements to the tool that could be made. A more automated setup approach, perhaps using one centralized setup script, would make the tool vastly easier to use. Additionally, distributing the tool in the form of a container would make it more portable and would ease the pain of dealing with so many dependencies.

While we improved the documentation greatly, certainly more could be done. Perhaps documentation on how to deal with common errors would be useful.

While we focused on integer division operations, there are other operations, such as floating point operations, that could lead to side-channel attacks and therefore may be worth detecting using CT-Prover.

Finally, further analysis of our detected tainted division operations is necessary. This may include using phases 2 and 3 of CT-Prover, but also manual analysis to discover if these potential vulnerabilities are exploitable in any way.

8 Conclusion

As has been shown, searching for timing-based side channels is extremely important, and even division operations have been shown to cause feasible remote timing attacks [2, 3, 5]. We have updated CT-Prover, a tool used to detect possible side-channel sources using static taint analysis, to consider

integer divisions with tainted operands as a new potential side-channel source. We found eight previously unknown tainted divisions across three different programs in CT-Prover's original test suite, and also verified the efficacy of our methods by showing how CT-Prover can now detect the tainted division that made KyberSlash possible [2].

Additionally, we have made CT-Prover vastly easier to use by adding helpful scripts, fixing code issues, and adding the capability to show the actual tainted source lines.

References

[1] ALMEIDA, J. B., BARBOSA, M., BARTHE, G., DUPRESSOIR, F., AND EMMI, M. Verifying {Constant-Time} implementations. In *25th USENIX Security Symposium (USENIX Security 16)* (2016), pp. 53–70.

[2] BERNSTEIN, D. J., BHARGAVAN, K., BHASIN, S., CHATTOPADHYAY, A., CHIA, T. K., KANNWISCHER, M. J., KIEFER, F., PAIVA, T., RAVI, P., AND TAMVADA, G. KyberSlash: Exploiting secret-dependent division timings in kyber implementations. Cryptology ePrint Archive, Paper 2024/1049, 2024.

[3] BRUMLEY, B. B., AND TUVERI, N. Remote timing attacks are still practical. In *European Symposium on Research in Computer Security* (2011), Springer, pp. 355–371.

[4] CAI, L., SONG, F., AND CHEN, T. Towards efficient verification of constant-time cryptographic implementations. *Proceedings of the ACM on Software Engineering 1*, FSE (2024), 1019–1042.

[5] SCHRÖDER, R. L., GAST, S., AND GUO, Q. Divide and surrender: Exploiting variable division instruction timing in {HQC} key recovery attacks. In *33rd USENIX Security Symposium (USENIX Security 24)* (2024), pp. 6669–6686.

[6] SCHUBERT, P. D., HERMANN, B., AND BODDEN, E. Phasar: An inter-procedural static analysis framework for c/c++. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (2019), Springer, pp. 393–410.

[7] SUI, Y., AND XUE, J. Svf: interprocedural static value-flow analysis in llvm. In *Proceedings of the 25th international conference on compiler construction* (2016), pp. 265–266.